

HASHMAP

"What you do today can
improve all your
tomorrows."

~ Ralph Marston



THE BRIGHT
DROPS
.com



Good

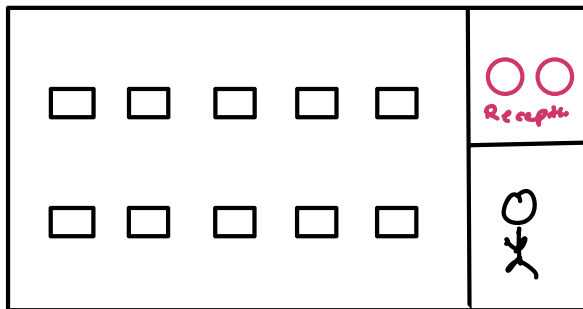
Evening

Today's content

- Hashmap Intro
- Frequency of each query
- Distinct ele
- Subarray with sum = 0

HashMap Introduction

Mayuresh Hotel → 10

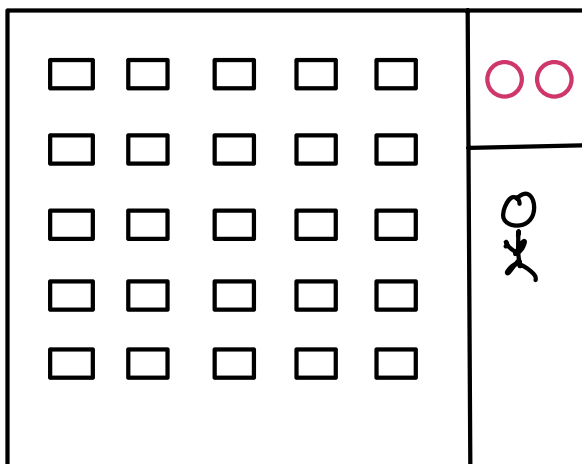


Anand → Room no. 5

Register

Room no	Availability
1	x
2	✓
3	x
4	x
5	x
⋮	⋮
10	✓

Mayuresh's new hotel → 1000



boolean array [1001]

↓

give information for
all the rooms

```
boolean [] checkIn = new boolean [1001];
```

↳ Initialised with false

means every room is available

check in [3] $\rightarrow$ mark it as True

check in [78] $\rightarrow$ make it as True

checkIn[78] = True:

To check if a particular room is available or not $\rightarrow$ $O(1)$ time complexity

Lucky numbers $\rightarrow [1 - 10^9]$

6	10	1000	7811	123	OO
☒	☒	☐	☒	☐	
10^9					O X
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	

For one on one mapping

boolean array = $10^9 + 1$ size

arr[6] = T

arr[7811] = T

arr[10] = T

arr[123] = F

arr[1000] = F

Is this a good way to store the info? NO

Issue $\rightarrow$ For 1000 rooms $\rightarrow 10^9 + 1$ space

$\hookrightarrow$ Wasting huge space

Advantage $\rightarrow$ Accessing of room availability $\rightarrow O(1)$

Data structure $\rightarrow$ HashMap

Accessing $\rightarrow O(1)$

Optimise the
space, for

1000 rooms $\rightarrow$ space = 1000

HashMap $\rightarrow$ Key & Value

occupied $\rightarrow$ True
Available $\rightarrow$ False

$\hookrightarrow$ Data in the form of Key & Value pair

$\langle 6, \text{occupied} \rangle$

$\langle 10, \text{occupied} \rangle$

$\langle 1000, \text{occupied} \rangle$

$\langle 7811, \text{occupied} \rangle$

TC for searching in
hashmap $\rightarrow O(1)$

irrespective of no. of
entries in the HashMap

size of hashmap $\rightarrow O(n)$ for storing n entries

Yogesh $\rightarrow$ 7811 $\rightarrow$ look for this room no.

$\downarrow$

Unique & can be used to
identify the room availability

Note:- Keys are unique &
Values may or maynot be unique.

Declaration ↘

```
HashMap<Key, Value> hm = new HashMap<>();
```

01. Store population of every country

key : Country Name - Unique {String}
value : Population {Long}

```
HashMap<String, Long> hm = new HashMap<>();
```

02. No. of states in each country

Key → Country name → String
value → No. of states → Integer

```
HashMap<String, Integer> hm = new HashMap<>();
```

03. Store the frequency of elements of arr

Key $\rightarrow$ unique element $\rightarrow$ Integer

Value $\rightarrow$ Count of element $\rightarrow$ Integer

arr = [2, 3, 1, 2, 3]
 0 1 2 3

HashMap < Integer, Integer > freq = new HashMap < > ();

2	$\rightarrow$	X 2
3	$\rightarrow$	X 2
1	$\rightarrow$	1

Rules in HashMap < key, value >

01. Keys $\rightarrow$ (a) Have to be unique

(b) Type of Key : $\left\{ \begin{array}{l} \text{Integer, Long, Float,} \\ \text{Double, String, Character} \end{array} \right\}$

02. Values $\rightarrow$ (a) They can repeat

(b) Type of value { It can be anything }

Q → For every country, we want to know the name of all states

Keys → Country name → String

Values → All state names

↓

ArrayList < String >

HashMap < String, ArrayList < String > > hm = ____;

Query → India → { Name of all states }

Q → For each country, give me the info of the population of each state

Key → Country name

Value → population of every state

HashMap < State, population >

HashMap < String, HashMap < State, population > > hm;

Most used functions in Hashmap

01. `size()` : return no. of keys present in hashmap
02. `put(key, Value)` : insert following key value pair in hashmap
03. `containsKey(key)` : Return T/F based on current key present.
04. `get(key)` : If key present in Java, Return value of key
If key not present, Return null
05. `remove(key)` : Remove the complete pair from HM

Note :-

01. Order $\rightarrow$ Not maintained for keys

Time complexity $\rightarrow$ If key is number/char/boolean

$$TC = O(1)$$

$\rightarrow$ If key is string of length l

$$TC = O(l)$$

Space complexity $\rightarrow$ For n entries $\rightarrow SC: O(n)$

HashSet

01. Only stores keys & keys has to be unique
02. Keys Datatype : Numbers / Decimal / String / Character

Declaration $\rightarrow$ `HashSet < Datatype > hs = new HashSet < > ();`

Functions

01. `size()` $\rightarrow$ return no. of keys present in hashset
02. `add(key)` $\rightarrow$ insert following key in hashset.
03. `contains(key)` $\rightarrow$ Return T/F based on key
04. `remove(key)` $\rightarrow$ Remove key from hashset

Note :-

Order $\rightarrow$ Is not maintained for keys

Time complexity $\rightarrow$ If key is number / char / boolean

$$TC = O(1)$$

$\rightarrow$ If key is string of length l

$$TC = O(l)$$

Space complexity $\rightarrow$ For n entries $\rightarrow$ $SC: O(n)$

01. Find frequency of numbers

Given $ar[N]$ elements & Q queries, for each query find frequency of each element in array.

Constraint:

$$1 \leq N, Q \leq 10^5$$

$$1 \leq ar[i] \leq 10^9$$

Ex: $ar[11] =$ 2 6 3 8 2 8 2 3 8 10 6

Queries: 4

2 $\rightarrow$ 3
8 $\rightarrow$ 3
3 $\rightarrow$ 2
5 $\rightarrow$ 0

Idea 1 $\rightarrow$ For each query, iterate on the array & get the count

$$TC = O(Q * N)$$

$$= O(10^5 * 10^5) = 10^{10} \} TLE$$

Idea 2 $\rightarrow$ Use hashmap

Key $\rightarrow$ Unique ele

Value $\rightarrow$ freq of unique ele

arr = 1 4 7 8 4 3 7 7

key - value

1	→	1
4	→	x 2
7	→	x x 3
8	→	1
3	→	1

Queries → 3

01. (6) → map.get(7) = 3
 02. (3) → map.get(3) = 1
 03. (2) → map.get(2) = 0

```
void printfreq (int [n] arr, int [q] Queries)
```

// Create a HM of key(unique ele) → value(freq)

```
HashMap < Integer, Integer > map = new HashMap<>();
```

```
for (int i=0; i<n; i++)
```

```
    int ele = arr[i];
```

```
    if (map.containsKey(ele) == true) {
```

```
        int of = map.get(ele)
```

```
        map.put(ele, of+1);
```

```
    }
```

```
    else {
```

```
        map.put(ele, 1);
```

```
    }
```

```
}
```

TC = O(n+q)

SC = O(n)

// Iterate on Q queries

```
for (i = 0 ; i < Q ; i++)
```

```
    int key = Queries[i];
```

```
        if (map.containsKey(key))
```

```
            | print (map.get(key));
```

```
            | }
```

```
        else {
```

```
            | print (0);
```

```
            | }
```

```
    }
```

```
}
```

0	1	2	3	4
1	2	3	2	1

ele = 1

```
for (int i=0; i<n; i++)
```

```
    int ele = ar[i];
```

```
    if (map.containsKey(ele) == true) {
```

```
        | int of = map.get(ele)
```

```
        | map.put (ele, of+1);
```

```
    }
```

```
    else {
```

```
        | map.put (ele, 1);
```

map

key value

1	→	1
2	→	2
3	→	1

02. Given $arr[N]$ ele, find no. of distinct elements

Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq arr[i] \leq 10^5$$

$$arr[5] = \underline{\underline{\text{3} \text{ 5 6 5 4}}} \rightarrow 4$$

$$arr[5] = \underline{\underline{1 \ 1 \ 1 \ 2 \ 2}} \rightarrow 2$$

$$arr[5] = \underline{\underline{\text{3} \text{ 3 3 3 3}}} \rightarrow 1$$

↓

$$\text{Hash set} = \{ 3 \}$$

Idea $\rightarrow$ Insert all the element in hs
& get the size

int distinct ele (int [] arr)

```
HashSet <Integer> hs = new HashSet<>();
```

```
for (i = 0; i < n; i++) {
```

```
    |   hs.add(arr[i]);
```

TC = $O(n)$

SC = $O(n)$

3

```
return hs.size();
```

}

03. Given arr[N], check if all elements are distinct or not

arr[5] = 3 5 6 5 4 → False

arr[3] = 1 2 3 → True

arr[5] = 3 5 6 1 8 → True

Idea → Insert all the ele in HashSet

if (hs.size() == arr.length) → True

else → False

04. Given $arr[N]$ elements, check if there exists a subarray with sum = 0

Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq arr[i] \leq 10^9$$

$$arr[5] : \begin{array}{c|ccccc} 0 & 1 & 2 & 3 & 4 \\ \hline 2 & 4 & -1 & -2 & 1 \\ \hline \end{array} \rightarrow \text{False}$$

$$arr[10] : \begin{array}{c|cccccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \hline 2 & 2 & 1 & -3 & 4 & 3 & 1 & -2 & -3 & 2 \\ \hline \end{array} \rightarrow \text{True}$$

$$arr[5] : \begin{array}{c|ccccc} 0 & 1 & 2 & 3 & 4 \\ \hline 2 & -1 & 0 & 2 & 3 \\ \hline \end{array}$$

subarray $\rightarrow$ single ele be a subarray $\checkmark$

Idea 1 $\rightarrow$ Generate all subarrays sum & compare == 0

01. Use 3 loops $TC = O(n^3)$

02. Using prefix sum $TC = O(n^2)$ $SC = O(n)$

03. Carry forward $TC = O(n^2)$ $SC = O(1)$

Max value of $N = 10^5$

$$O(n^2) = 10^{10} \rightarrow \text{TLE}$$

Idea 2 →

	0	1	2	3	4	5	6	7	8	9
arr[10]	2	2	1	-3	4	3	1	-2	-3	2
psum[10]	2	4	5	2	6	9	10	8	5	7

In above case

$$psum[2] = sum[0 \ 2] = 5$$

$$psum[8] = sum[0 \ 8] = 5$$

$$sum[0 \ 8] = sum[0 \ 2] + sum[3 \ 8]$$

$$5 = 5 + sum[3 \ 8]$$

$$5 - 5 = sum[3 \ 8]$$

$$sum[3 \ 8] = 0$$

—h—h—

$$psum[0] = sum[0 \ 0] = 2$$

$$psum[3] = sum[0 \ 3] = 2$$

$$sum[0 \ 3] = sum[0 \ 0] + sum[1 \ 3]$$

$$2 = 2 + sum[1 \ 3]$$

$$sum[1 \ 3] = 0$$

Edge Case → 2 -5 3 6

psum → 2 -3 0 6

} Data is not repeating but the subarray sum = 0 exists

Idea → If psum contains zero → return true

boolean subarrzero (int [] arr)

long psum[] = new int [n];

HashSet < Long > hs = new HashSet<>();

psum[0] = arr[0]

hs.add (psum[0]); ←

TC = $O(n)$

SC = $O(n+n)$

for (i=1; i<n; i++) {

≈ $O(n)$

 psum[i] = psum[i-1] + arr[i];

 hs.add (psum[i]);

3

if (hs.size() < n || hs.contains(0)) return true

else return false

}

	0	1	2	3	4	5	6	7	8	9
arr[10] :	2	2	1	-3	4	3	1	-2	-3	2
psum[10]	2	4	5	2	6	9	10	8	5	7

hs = {2, 4, 5}

for (i=1; i<n; i++)

psum[i] = psum[i-1] + arr[i] → ~~4~~ 2

if (psum[i] == 0) return true;

if (hs.contains(psum[i])) → return true;

else hs.add(psum[i])

3



Another approach

Checking hs before inserting

Doubt session

$$A \% M = B \% M \quad \text{max value of } M$$

$$A \% M - B \% M = 0$$

$$A \% M - B \% M + 0 = 0$$

$$A \% M - B \% M + M \% M = 0$$

$$\underbrace{\hspace{10em}}$$

$$(A-B) \% M = 0$$

return A-B

Backtracking / Greedy → Next class